



PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

Sistema de Gestión y Asignación de Turnos de Enfermería

Integrantes:

Rodrigo Escudero

Ariel Fuenzalida

Matías Santelices

11 de septiembre de 2026

Índice

Lista de figuras	3
Lista de tablas	4
Introducción	5
1 ¿Qué datos maneja el sistema? (SIA-1)	6
2 Las clases que construyen el sistema (SIA-2)	7
2.1 Clase Persona	7
2.2 Clase Enfermera	7
2.3 Jerarquía de turnos	7
2.4 Clase AreaHospitalaria	7
3 Buenas prácticas (SIA-3)	9
4 Uso de colecciones de Java (SIA-4)	10
4.1 Primer nivel: registro global de enfermeras	10
4.2 Segundo nivel: historial de turnos	10
4.3 Relación de anidación	10
5 Sobrecarga de métodos (SIA-5)	12
5.1 Clase AreaHospitalaria	12
5.2 Clase TurnoControlador	12
6 Sobreescritura y polimorfismo (SIA-6)	13
6.1 Uso del polimorfismo	13
7 Funcionalidades básicas del menú (SIA-7)	14
7.1 Modo Consola	14
7.2 Modo Gráfico	14
8 Editar, eliminar y buscar (SIA-8)	16
8.1 Búsqueda	16
8.2 Edición	16
8.3 Eliminación	16
8.4 Operaciones en ventanas	16
9 Funcionalidad propia del programa (SIA-9)	17
9.1 Validación de disponibilidad para una nueva asignación	17
9.2 Consultas complementarias	17
10 Consola o ventanas, a elección (SIA-10)	19
11 Persistencia de datos al iniciar y salir (SIA-11)	20

11.1 Carga inicial	20
11.2 Guardado al salir	20
12 Manejo de excepciones (SIA-12)	21
12.1 Captura y comunicación de errores	21
12.2 Ejemplo: registro con RUT inválido	21
13 Requisitos opcionales (SIA-O1 y SIA-O4)	22
13.1 Gráficos estadísticos (SIA-O1)	22
13.2 Modelo-Vista-Controlador (SIA-O4)	22
14 UML completo del programa	23
15 Control de versiones con GitHub (SIA-13)	24

Índice de figuras

Figura 2.1	Diagrama de clases del sistema de turnos	8
Figura 4.1	Estructura de las colecciones principales: registro global e historiales de turnos	11
Figura 7.1	Vista previa del menú en modo ventana	15
Figura 9.1	Estadísticas de horas por enfermera y distribución de tipos de evento	18
Figura 10.1	Selección del modo de interfaz al iniciar la aplicación	19
Figura 14.1	Diagrama UML completo del programa	23
Figura 15.1	Evidencia del historial de commits del repositorio	24

Índice de tablas

Tabla 12.1	Excepciones personalizadas del sistema		21
------------	--	-------	----

Introducción

El presente informe detalla el diseño, desarrollo e implementación del **Sistema de Gestión y Asignación de Turnos de Enfermería**, una aplicación construida utilizando el lenguaje de programación Java bajo el paradigma de Orientación a Objetos.

En el entorno hospitalario, la correcta asignación de turnos, el manejo de licencias y el seguimiento de horas trabajadas por el personal de enfermería son tareas críticas. Este sistema busca solucionar la gestión manual de esta información, proveyendo un software capaz de almacenar, filtrar y persistir datos eficientemente a través del patrón de arquitectura Modelo-Vista-Controlador (MVC).

A lo largo del documento, se exploran las decisiones técnicas adoptadas para cumplir con los requerimientos del Sistema de Información Académica (SIA) de la asignatura Programación Avanzada, destacando la implementación de Colecciones, Polimorfismo, Manejo de Excepciones y Persistencia.

1. ¿Qué datos maneja el sistema? (SIA-1)

El sistema permite administrar al personal de enfermería y organizar sus turnos por área hospitalaria. Maneja los siguientes datos:

- **Enfermera:** registra RUT, nombre, apellidos, edad, especialidad y área asignada. El RUT identifica a cada enfermera dentro del registro global.
- **Turno:** registra identificador, fecha, hora de inicio, hora de término y observación. Existen tres variantes: turno regular, licencia y cambio de turno.
- **Datos específicos de cada evento:** el turno regular indica el tipo de jornada; la licencia incorpora motivo y tipo de licencia; el cambio de turno identifica mediante RUT a la sustituta y registra el motivo del cambio.
- **Área hospitalaria:** permite agrupar al personal según su área asignada para realizar consultas y asignaciones grupales.

El registro global contiene los objetos Enfermera y cada uno mantiene su propia lista de turnos. Las agrupaciones por área se obtienen a partir de ese registro.

Las funcionalidades principales permiten agregar, listar, buscar, editar y eliminar enfermeras y turnos. También permiten asignar turnos por área, consultar estadísticas, filtrar por cantidad de turnos y comprobar la disponibilidad de personal antes de una nueva asignación.

La comprobación de disponibilidad considera los eventos propios de cada enfermera y las sustituciones que realiza, para evitar asignaciones con horarios incompatibles.

2. Las clases que construyen el sistema (SIA-2)

Las clases del dominio representan a las enfermeras, sus eventos y las agrupaciones por área hospitalaria.

2.1. Clase Persona

Almacena nombre, apellido paterno, apellido materno, RUT y edad. Incluye métodos de acceso a estos datos y operaciones para normalizar y validar el RUT.

2.2. Clase Enfermera

Hereda de `Persona` y añade los atributos `especialidad`, `areaAsignada` y `listaTurnos`.

La colección `listaTurnos` es un `ArrayList<Turno>` que contiene el historial de la enfermera. La clase permite agregar, buscar, editar y eliminar eventos, además de consultar cantidades de turnos y horas de sus turnos regulares.

2.3. Jerarquía de turnos

`Turno` es una clase abstracta. Define los atributos comunes `id`, `fecha`, `horaInicio`, `horaFin` y `observacion`, junto con sus métodos de acceso.

Sus tres subclases son:

- **TurnoRegular:** incorpora `tipoTurno`, que identifica la jornada de mañana, tarde o noche.
- **Licencia:** incorpora `motivo` y `tipoLicencia`. El motivo se mantiene sincronizado con la observación del evento.
- **CambioTurno:** incorpora `rutSustituta` y `motivoCambio`. La sustituta se identifica mediante su RUT.

Las tres subclases implementan los métodos abstractos `getResumen()` y `getTipo()`. Esto permite recorrer el historial utilizando referencias de tipo `Turno` y obtener la descripción correspondiente a cada evento.

2.4. Clase AreaHospitalaria

Representa una agrupación de personal por área. Contiene los atributos `nombre`, `minimoEnfermeras` y un mapa auxiliar `TreeMap<String, Enfermera>` denominado `enfermeras`.

Permite agrupar y listar al personal del área y comprobar si su cantidad alcanza el mínimo definido. Esta comprobación de cantidad es distinta de la validación de disponibilidad por fecha y horario realizada por `TurnoControlador`.

El registro global del sistema se mantiene en **Main**. El mapa de **AreaHospitalaria** reúne referencias a las enfermeras de ese registro; no sustituye a la colección principal.

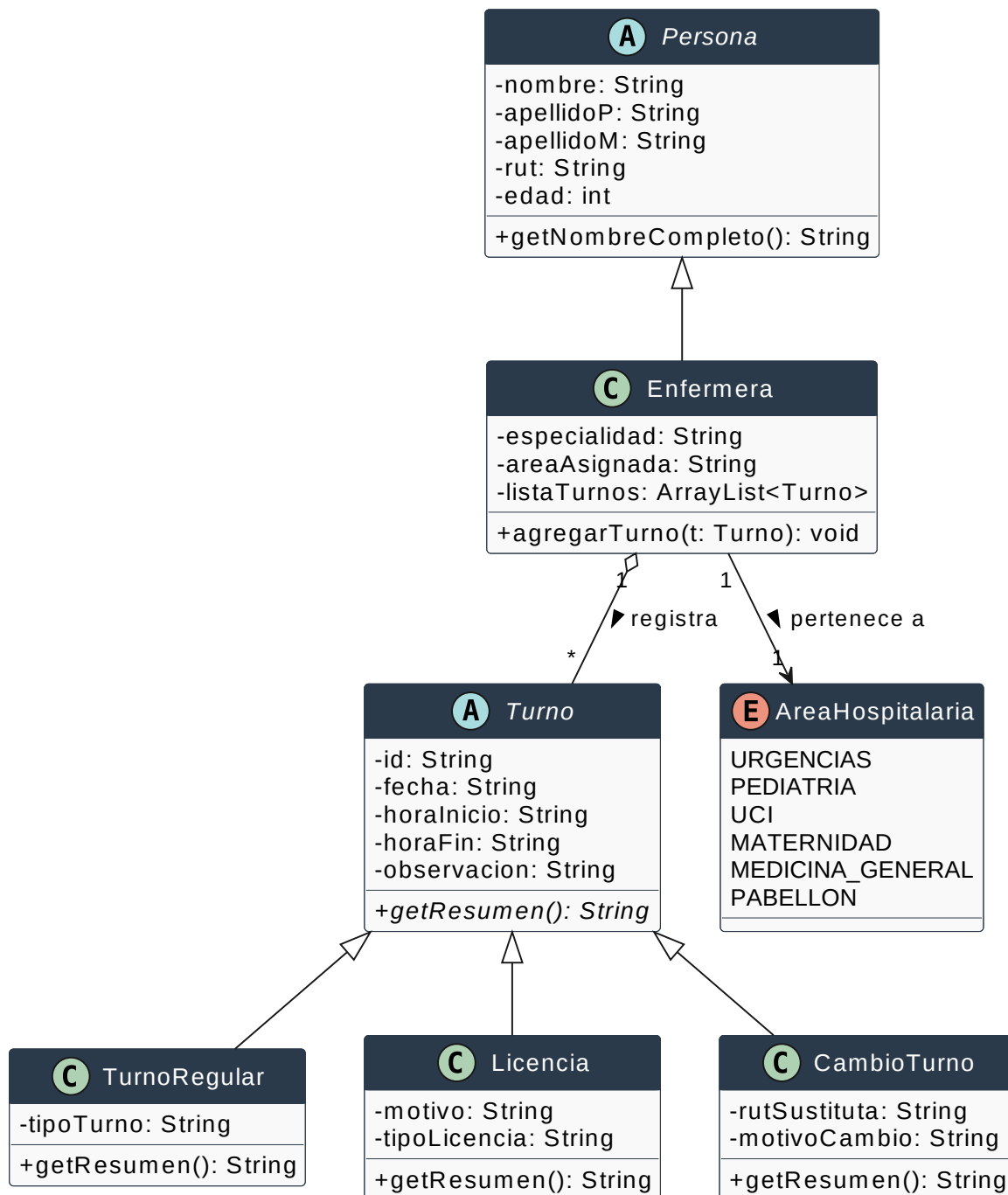


Figura 2.1: Diagrama de clases del sistema de turnos

3. Buenas prácticas (SIA-3)

El programa aplica las siguientes prácticas para facilitar la lectura, el mantenimiento y la protección de los datos:

- **Encapsulamiento:** los atributos propios de las clases se declaran como `private` y cuentan con métodos de lectura y escritura. Las validaciones de datos, como el RUT y la edad, permiten rechazar valores inválidos.
- **Código modular:** las clases se organizan en paquetes de modelo, controlador y vista. Las ventanas utilizan Swing, los controladores reúnen operaciones compartidas y `GestorArchivos` concentra la persistencia. `Main` contiene el inicio de la aplicación y el menú de consola.
- **Documentación interna:** los comentarios describen la finalidad de las clases, las operaciones y las reglas relevantes, como la comprobación de conflictos de horario y el registro de sustituciones.
- **Datos iniciales en el código:** `GestorArchivos` incluye el método `cargarDatosIniciales()`, que crea seis enfermeras y eventos de ejemplo de septiembre de 2026. Estos incluyen turnos regulares, licencias y cambios de turno, distribuidos entre distintas áreas.
- **Inicio con información disponible:** cuando no existen los archivos CSV, se utilizan los datos iniciales definidos en el código. Si los archivos existen, se recuperan los datos guardados de la sesión anterior. Los ejemplos iniciales permiten probar las operaciones sin registrar primero al personal de forma manual.

4. Uso de colecciones de Java (SIA-4)

El sistema utiliza dos colecciones principales del Java Collections Framework: un mapa de enfermeras y una lista de turnos dentro de cada enfermera.

4.1. Primer nivel: registro global de enfermeras

La clase `Main` mantiene el atributo `registroGlobal`, de tipo `TreeMap<String, Enfermera>`.

La **clave** es el RUT normalizado y el **valor** es una referencia al objeto `Enfermera`. El mapa permite consultar los registros por su clave y mantiene las claves ordenadas según el orden natural de `String`.

4.2. Segundo nivel: historial de turnos

Cada objeto `Enfermera` contiene el atributo `listaTurnos`, de tipo `ArrayList<Turno>`.

Esta lista almacena objetos de las clases `TurnoRegular`, `Licencia` y `CambioTurno`, utilizando su tipo común `Turno`.

El `ArrayList` conserva el orden de inserción de los eventos. No los ordena automáticamente por fecha.

4.3. Relación de anidación

Los valores del mapa principal son objetos `Enfermera`, y cada uno contiene su propia lista de turnos. Por tanto, la segunda colección está anidada dentro de los elementos de la primera.

El mapa de `AreaHospitalaria` y las listas utilizadas para devolver resultados filtrados son colecciones auxiliares. Permiten agrupar o consultar los datos y no sustituyen al registro global ni a los historiales individuales.

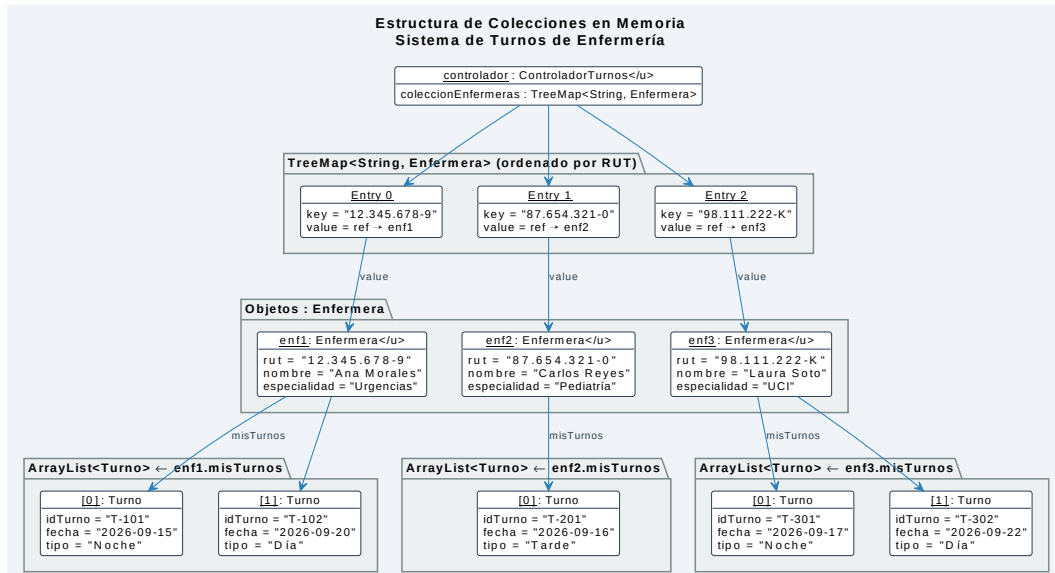


Figura 4.1: Estructura de las colecciones principales: registro global e historiales de turnos

5. Sobrecarga de métodos (SIA-5)

La sobrecarga consiste en declarar métodos con el mismo nombre y distintas listas de parámetros. El compilador determina qué variante utilizar según los argumentos de la llamada.

El programa utiliza sobrecarga en las siguientes clases:

5.1. Clase AreaHospitalaria

- `listarEnfermeras()`: muestra las enfermeras que forman parte de la agrupación por área.
- `listarEnfermeras(String tipoTurno)`: muestra las enfermeras del área que tienen turnos regulares del tipo indicado.

Ambas variantes se utilizan en la consola. La asignación grupal muestra al personal del área mediante la versión sin parámetros, mientras que el resumen por área permite consultar al personal según el tipo de turno seleccionado.

5.2. Clase TurnoControlador

- `registrar(Enfermera enfermera, Turno turno)`: registra el evento utilizando el registro global de la aplicación.
- `registrar(Enfermera enfermera, Turno turno, Map<String, Enfermera>registro)`: recibe explícitamente el registro sobre el que debe validar y agregar el evento.

La primera variante se utiliza desde las interfaces de consola y ventanas. Esta delega en la segunda variante, que también se utiliza directamente durante la carga de archivos CSV y la creación de los datos iniciales.

La sobrecarga permite reutilizar las validaciones de registro en estos distintos contextos. Los ejemplos corresponden a métodos regulares, no a constructores.

6. Sobreescritura y polimorfismo (SIA-6)

La clase abstracta `Turno` declara los métodos `getResumen()` y `getTipo()`. Las clases `TurnoRegular`, `Licencia` y `CambioTurno` implementan estos métodos utilizando la anotación `@Override`.

- **TurnoRegular:** entrega un resumen de la jornada, su fecha y horario.
- **Licencia:** entrega un resumen de la licencia con los datos correspondientes al permiso.
- **CambioTurno:** entrega un resumen del cambio, incluyendo información sobre la sustituta y el motivo.

El método `getTipo()` identifica la categoría del evento: regular, licencia o cambio de turno.

6.1. Uso del polimorfismo

La colección `listaTurnos` almacena referencias de tipo `Turno`, aunque los objetos pertenecen a sus distintas subclases.

Al consultar el historial, el programa recorre esta colección e invoca `getResumen()` sobre cada elemento. Java ejecuta la implementación correspondiente al tipo concreto del objeto.

Por tanto, no es necesario seleccionar manualmente una implementación de resumen según la clase del evento. Las tres subclases se utilizan en los datos iniciales y pueden registrarse desde las interfaces del programa.

7. Funcionalidades básicas del menú (SIA-7)

El sistema permite insertar y listar elementos de las dos colecciones principales mediante opciones separadas, tanto en consola como en ventanas.

7.1. Modo Consola

El menú principal incluye las siguientes opciones de inserción y listado:

- **Opción 1 - Agregar Enfermera:** solicita los datos personales y laborales, valida el RUT y la edad, y registra a la enfermera si su RUT no existe en el registro global.
- **Opción 2 - Listar Enfermeras:** muestra las enfermeras del registro global y sus datos.
- **Opción 6 - Registrar Turno:** permite seleccionar una enfermera e ingresar un turno regular, una licencia o un cambio de turno. Antes de agregar el evento se comprueban las validaciones correspondientes.
- **Opción 7 - Ver Historial de Turnos:** muestra los eventos registrados en la lista de la enfermera seleccionada, incluyendo sus identificadores y resúmenes.

El historial conserva el orden de inserción de los eventos; no se ordena automáticamente por fecha.

7.2. Modo Gráfico

- **Gestión de enfermeras:** la ventana `VentanaEnfermeras` presenta una tabla con los registros. El botón de agregar abre el formulario de ingreso y la opción de mostrar todas permite recuperar el listado completo después de una búsqueda.
- **Gestión de turnos:** la ventana `VentanaTurnos` presenta el historial de una enfermera. El botón de agregar permite elegir el tipo de evento y completar su formulario. La opción de mostrar todos actualiza el listado de sus eventos.

Las tablas muestran los registros existentes. La disponibilidad para una fecha y horario específicos se consulta mediante la función de validación de disponibilidad.

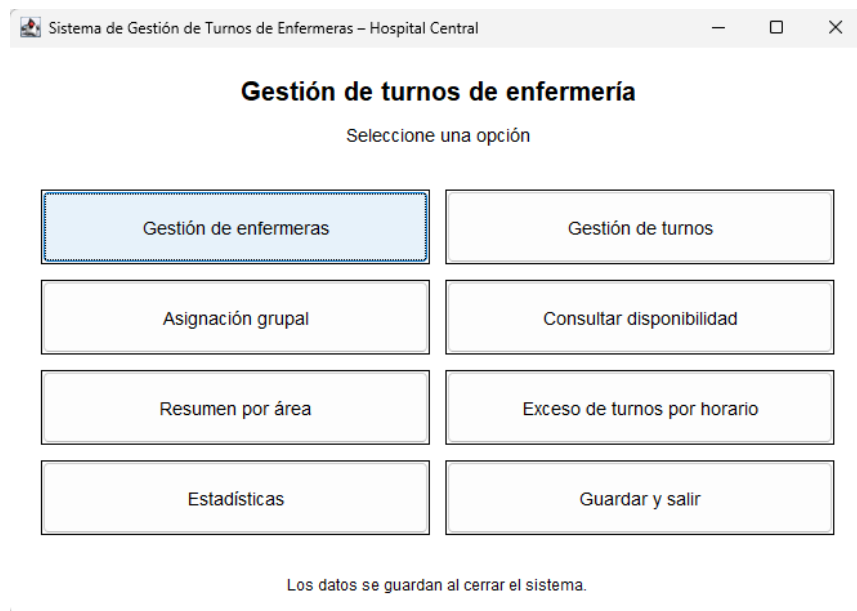


Figura 7.1: Vista previa del menú en modo ventana

8. Editar, eliminar y buscar (SIA-8)

El sistema ofrece operaciones de búsqueda, edición y eliminación para ambas colecciones. En consola se presentan mediante opciones separadas del menú principal; en ventanas se ejecutan desde los controles de gestión de enfermeras y turnos.

8.1. Búsqueda

- **Enfermeras, opción 3:** permite buscar por RUT o nombre. La búsqueda por RUT consulta el registro global mediante su clave normalizada, mientras que la búsqueda por nombre permite obtener coincidencias entre las enfermeras registradas.
- **Turnos, opción 8:** permite buscar un evento por su identificador en los historiales del registro global e informar a qué enfermera pertenece.

8.2. Edición

- **Enfermeras, opción 4:** permite modificar nombre, apellidos, edad, especialidad y área asignada. El RUT se mantiene como identificador del registro y la enfermera conserva su historial.
- **Turnos, opción 9:** permite modificar la observación de un evento. Cuando se trata de una licencia, su motivo se mantiene sincronizado con la observación.

8.3. Eliminación

- **Enfermeras, opción 5:** permite eliminar una enfermera y su historial, previa confirmación. La operación se rechaza si figura como sustituta en cambios de turno de otras enfermeras.
- **Turnos, opción 10:** permite eliminar un evento del historial de la enfermera seleccionada, previa confirmación. Las consultas posteriores de cantidades y horas se calculan a partir de los eventos que permanecen registrados.

Eliminar un cambio de turno no restaura automáticamente el turno regular que pudo haber reemplazado.

8.4. Operaciones en ventanas

`VentanaEnfermeras` permite buscar registros y seleccionar una enfermera para editarla o eliminarla. `VentanaTurnos` permite buscar eventos por ID, editar la observación del evento seleccionado y eliminarlo.

Las operaciones utilizan los controladores del sistema y presentan mensajes para informar los resultados o las condiciones que impiden completar una acción.

9. Funcionalidad propia del programa (SIA-9)

9.1. Validación de disponibilidad para una nueva asignación

Esta funcionalidad permite comprobar si existe suficiente personal disponible antes de realizar una nueva asignación.

El usuario indica el área hospitalaria, la fecha, la hora de inicio, la hora de término y la cantidad de enfermeras necesarias.

El método `validarFactibilidadCobertura()` de `TurnoControlador` realiza las siguientes comprobaciones:

1. Valida los datos de la consulta.
2. Selecciona las enfermeras que pertenecen al área indicada.
3. Comprueba los eventos de cada enfermera, incluyendo las sustituciones que realiza y que están registradas en los historiales de otras enfermeras.
4. Cuenta al personal disponible y compara esa cantidad con la solicitada.

La comprobación considera los conflictos de horario, las licencias y los turnos que terminan al día siguiente.

El resultado indica si hay suficientes enfermeras disponibles. Esta operación procesa un subconjunto filtrado por área y no registra ni modifica eventos. Está disponible en la opción 14 de la consola y en la interfaz gráfica.

9.2. Consultas complementarias

El sistema también permite filtrar enfermeras que superan una cantidad de turnos regulares de mañana, tarde o noche durante un mes y año determinados. El límite corresponde a una cantidad de turnos, no a una cantidad de horas.

La ventana de estadísticas presenta las horas trabajadas por enfermera y la distribución de eventos entre turnos regulares, licencias y cambios. Las horas de las sustituciones se atribuyen a la enfermera que cubre el evento.

Estos gráficos complementan las consultas del sistema y corresponden al requisito opcional SIA-O1.

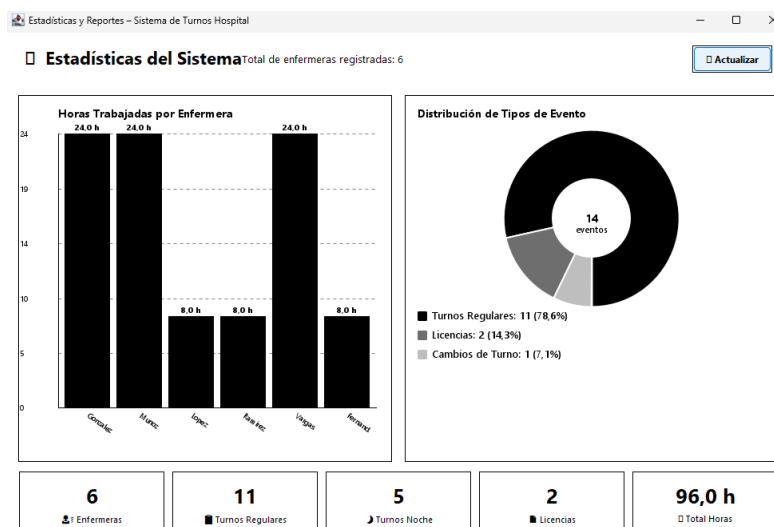


Figura 9.1: Estadísticas de horas por enfermera y distribución de tipos de evento

10. Consola o ventanas, a elección (SIA-10)

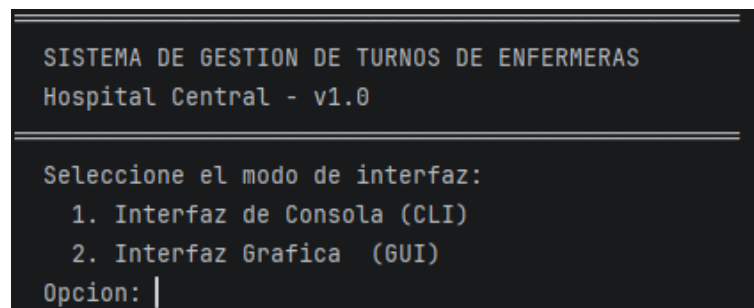
Al iniciar la aplicación, la clase **Main** carga los datos y solicita al usuario seleccionar el modo de interfaz:

- **Opción 1 - Consola:** utiliza un menú numerado y recibe las entradas mediante **Scanner**. Los resultados y mensajes se muestran en la terminal.
- **Opción 2 - Interfaz gráfica:** abre las ventanas construidas con Swing. Utiliza componentes como **JFrame**, **JPanel**, **JTable** y **JOptionPane**, con una presentación en blanco, negro y tonos neutros.

Ambas interfaces permiten agregar, listar, buscar, editar y eliminar enfermeras y turnos. También permiten realizar asignaciones grupales, consultar filtros y resúmenes por área, validar disponibilidad y consultar estadísticas.

Las interfaces utilizan los controladores y las clases del modelo para operar sobre los mismos datos en memoria. La consola presenta las estadísticas como texto y la interfaz gráfica añade gráficos.

Al salir, se ejecuta el guardado de los datos. Si el guardado falla, se informa al usuario y se permite reintentar.

A screenshot of a terminal window with a dark background and light-colored text. The text is as follows:

```
SISTEMA DE GESTION DE TURNOS DE ENFERMERAS
Hospital Central - v1.0

Seleccione el modo de interfaz:
  1. Interfaz de Consola (CLI)
  2. Interfaz Grafica (GUI)
Opcion: |
```

Figura 10.1: Selección del modo de interfaz al iniciar la aplicación

11. Persistencia de datos al iniciar y salir (SIA-11)

El sistema utiliza persistencia por lotes mediante archivos CSV. Los datos se cargan al iniciar la aplicación, se mantienen en memoria durante la sesión y se guardan al salir.

11.1. Carga inicial

La clase `Main` solicita la carga mediante `GestorArchivos.cargarEnfermeras()`.

Los archivos `enfermeras.csv` y `turnos.csv` se encuentran en la carpeta `resources/`, relativa al directorio de trabajo del programa.

Primero se recuperan las enfermeras y después se incorporan sus eventos. El registro de los turnos utiliza las validaciones del controlador, incluyendo la comprobación de conflictos de horario y sustituciones.

Cuando no existe ninguno de los dos archivos, se utilizan los datos iniciales definidos en el código. Si falta uno de los archivos o la carga presenta errores, se cancela el inicio para evitar trabajar con un registro incompleto y sobrescribir los datos originales.

11.2. Guardado al salir

Al solicitar la salida desde consola o ventanas, se invoca `GestorArchivos.guardarEnfermeras(...)` para escribir el registro completo.

Los CSV utilizan codificación UTF-8, punto y coma como separador y campos entre comillas para admitir textos con separadores, comillas y saltos de línea.

Antes de reemplazar los archivos existentes, el sistema prepara ambos archivos temporales y crea copias de respaldo con extensión `.bak`. Si ocurre un error durante la sustitución, intenta restaurar los archivos anteriores.

El método devuelve un resultado que indica si el guardado se completó. Si falla, la interfaz informa el problema y mantiene el sistema abierto para permitir un nuevo intento.

Las operaciones de agregar, editar y eliminar modifican las colecciones en memoria; no guardan individualmente los archivos. De esta forma se mantiene el funcionamiento por lotes solicitado.

12. Manejo de excepciones (SIA-12)

El sistema define dos excepciones personalizadas que heredan de `Exception`. Se utilizan para comunicar errores de validación y condiciones que impiden completar una operación.

Excepción	Uso en el programa
<code>RutInvalidoException</code>	Se lanza cuando el RUT no cumple las validaciones de formato o dígito verificador. Conserva el RUT ingresado para informar el error.
<code>TurnoConflictoException</code>	Comunica conflictos de horario y otras condiciones inválidas al registrar eventos o consultar disponibilidad, como una sustituta no registrada o una cantidad requerida no válida. Conserva una descripción del conflicto.

Tabla 12.1: Excepciones personalizadas del sistema

12.1. Captura y comunicación de errores

Las operaciones utilizan bloques `try-catch` para capturar las excepciones y comunicar su causa mediante mensajes en consola o cuadros de diálogo.

También se manejan errores de conversión numérica, validación de fechas y horarios, y lectura o escritura de archivos.

Un conflicto de asignación provoca el rechazo del evento solicitado. Si la carga de archivos queda incompleta, el inicio se cancela de forma controlada para conservar los archivos originales. Si el guardado falla, se informa al usuario y se permite reintentar la salida.

12.2. Ejemplo: registro con RUT inválido

1. El usuario ingresa los datos de una enfermera con un RUT inválido.
2. La validación del modelo detecta el problema y lanza `RutInvalidoException`.
3. El bloque `catch` de la operación captura la excepción.
4. La interfaz muestra un mensaje que informa el problema con el RUT.
5. La enfermera no se incorpora al registro y el usuario puede volver a realizar la operación con datos corregidos.

13. Requisitos opcionales (SIA-O1 y SIA-O4)

El sistema incorpora gráficos estadísticos y una organización del código basada en la separación de responsabilidades entre modelo, vista y controlador.

13.1. Gráficos estadísticos (SIA-O1)

La clase `VentanaEstadisticas` presenta dos gráficos dibujados mediante las herramientas gráficas del JDK:

- **Gráfico de barras:** muestra las horas trabajadas por enfermera. El cálculo considera sus turnos regulares y los cambios que cubre como sustituta.
- **Gráfico circular:** muestra la distribución de los eventos registrados entre turnos regulares, licencias y cambios de turno.

Los gráficos se construyen a partir de los datos en memoria y complementan los indicadores numéricos de la ventana. No representan la distribución del personal por especialidad.

13.2. Modelo-Vista-Controlador (SIA-O4)

El código se organiza en los siguientes paquetes:

- **Modelo:** contiene las entidades del dominio, como `Persona`, `Enfermera` y la jerarquía de `Turno`, junto con clases de apoyo para validaciones y persistencia.
- **Vista:** contiene las ventanas Swing y los métodos de presentación de la interfaz gráfica.
- **Controlador:** contiene `EnfermeraControlador` y `TurnoControlador`, que reúnen operaciones compartidas por las interfaces.

Las ventanas y las opciones de consola utilizan los controladores para ejecutar las operaciones de gestión. Los controladores trabajan con los objetos del modelo y aplican las comprobaciones correspondientes.

En esta implementación, `Main` también mantiene el registro global y contiene el menú de consola. `GestorArchivos` concentra la carga y el guardado de los datos.

15. Control de versiones con GitHub (SIA-13)

El proyecto utiliza Git y GitHub para mantener el historial de cambios del código fuente.

Durante el desarrollo se utilizaron ramas para trabajar en correcciones y mejoras. La versión de entrega se integró en la rama `main`, conservando el historial de los cambios incorporados.

El repositorio contiene más de seis commits, cumpliendo la cantidad mínima solicitada. El historial permite consultar los cambios registrados durante el desarrollo.

El código y su historial se encuentran disponibles en:

<https://github.com/YungRodri/Borrador-proyecto-enfermeras>

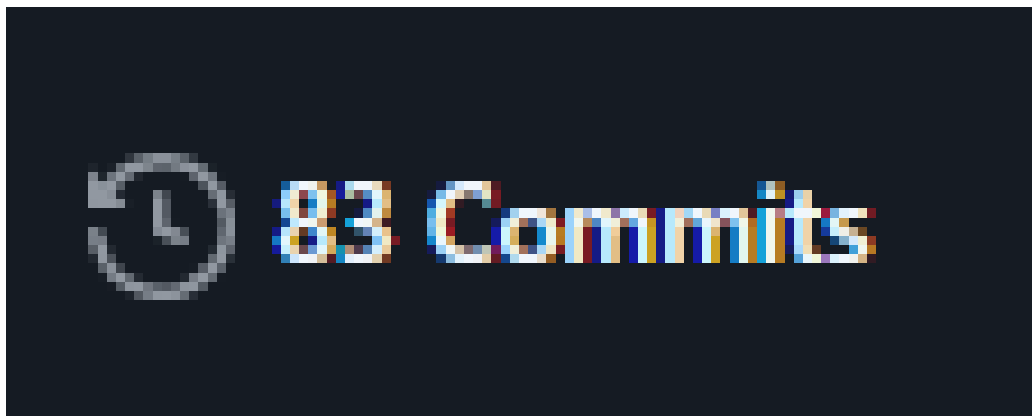


Figura 15.1: Evidencia del historial de commits del repositorio